

Forms and Static Files

INTRO

Q: 什么是Form?

A: Form (表单) 是一种用户交互界面, 用于收集用户输入的数据并发送到服务器进行处理。 ≠ table

表单的主要特性

1. **用户交互**: 表单是用户输入信息的主要方式, 比如输入用户名、密码、电子邮件地址或留言。
2. **数据传递**: 表单可以将用户输入的数据通过请求 (通常是 HTTP 请求) 发送到服务器, 比如通过 POST 或 GET 方法。
3. **动态处理**: 表单可以配合后端程序动态处理用户输入, 比如验证输入是否符合要求、存储数据或返回结果。

Forms

- Forms are very common and very complicated to implement correctly.
 - Any time you are accepting user input there are
 - security concerns (XSS Attacks),
 - proper error handling is required, and
 - there are UI considerations around how to alert the user to problems with the form.
 - Not to mention the need for redirects on success.
 - Fortunately, Django provides a rich set of tools to handle common use cases working with forms.
-

Django forms vs HTML forms

Introducing forms.py

- Django提供内置表单库--[django.forms](#) : handles HTML form display with **validation**. 用于处理 HTML 表单的显示以及表单数据的验证。
- The primary way to use the forms framework is to define a Form class for each HTML <form> you're dealing with.

使用 Django 的表单框架的主要方法是为每个 HTML 表单定义一个 **Form** 类。

这个 Form 类可以看作是表单的逻辑表示, 其中每个字段 (如用户名、密码) 都是类的一个属性。

- By convention, keep this Form class in a file called [forms.py](#), in the **same directory as your**

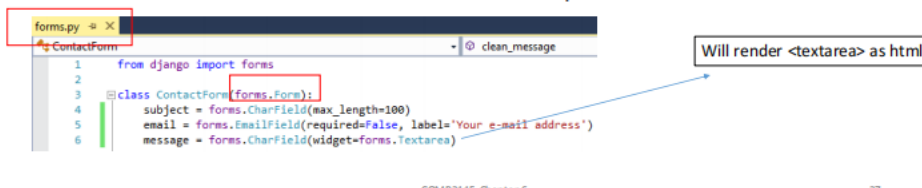
models.py

按照 Django 的约定，开发者通常将所有表单相关的类都放在一个名为 `forms.py` 的文件中。这个文件应该和 `models.py` 文件位于同一目录下（通常是在每个应用的目录中），以便代码结构清晰可管理。

```
Python |  
▼  
1  # forms.py  
2  from django import forms  
3  class LoginForm(forms.Form): //表单类-生成登陆页面的表单  
4      username = forms.CharField(max_length=100)  
5      password = forms.CharField(widget=forms.PasswordInput)
```

forms.py - forms.Form

- On slide 5, in our class-based BlogCreateView, we have used the generic CreateView to present a form to do a database insertion on submission of a valid form.
- We can build your form that is **not hooked to a database** with `forms.Form`
- The ContactForm shown below is an example.



Django 表单可以与数据库结合使用，也可以独立于数据库工作。

如果表单不需要直接与数据库交互，可以使用 `forms.Form` 类来创建自定义表单。（如果表单需要直接与数据库交互，在 Django 中可以使用 `ModelForm`）

“不需要直接与数据库交互”的表单是指那些不需要将用户输入的数据直接存储到数据库中的表单。

Django 表单与视图的结合

Tying form objects into views

Form is valid if all the required fields contain data.

A cleaned_data is a dictionary of the submitted data, "cleaned up" by converting values to the appropriate Python types.

This is an empty form. You can later try the following to note the difference:
`form = ContactForm(initial={'subject': 'I like your site!'})`

```
# views.py

from django.shortcuts import render
from mysite.forms import ContactForm
from django.http import HttpResponseRedirect
from django.core.mail import send_mail

# ...

def contact(request):
    if request.method == 'POST':
        form = ContactForm(request.POST)
        if form.is_valid():
            cd = form.cleaned_data
            send_mail(
                cd['subject'],
                cd['message'],
                cd.get('email', 'noreply@example.com'),
                ['siteowner@example.com'],
            )
            return HttpResponseRedirect('/contact/thanks/')
        else:
            form = ContactForm()
            return render(request, 'contact_form.html', {'form': form})
```

We will use POST method to submit data from the client.

COMP2115-Chapter 6

28

如何将 Django 表单对象与视图（views）结合使用？特别是在处理用户输入数据和执行操作方面。

- 使用视图函数 `contact(request)` 来处理表单的提交逻辑。
- 如果请求方法为 `POST`（表单提交），视图会创建一个表单实例，并将提交的数据传入：`form = ContactForm(request.POST)`。
- 1. 表单验证:
 - 调用 `form.is_valid()` 方法检查表单是否有效。
 - 表单有效的条件是所有必填字段都已填写且数据格式正确。
- 2. 清理后的数据（`cleaned_data`）：
 - 如果表单有效，可以通过 `form.cleaned_data` 访问经过验证和清理后的数据。这些数据会转换成适合 Python 类型的格式，比如 `email` 会转换成字符串类型。
- 3. 发送邮件:
 - 示例代码使用 `send_mail()` 函数发送邮件，邮件的内容来自表单的提交数据。
 - 具体逻辑包括：主题（subject）、正文（message）、发送者地址（email），以及接收者地址。
- 4. 提交后的重定向:
 - 表单提交成功后，用户会被重定向到一个“感谢页面”：`HttpResponseRedirect('/contact/thanks/')`。
 - 这种重定向提高了用户体验，避免重复提交表单。
- 5. 处理非 POST 请求（空表单）：
 - 如果请求方法不是 `POST`（例如用户初次访问表单页面），视图会创建一个空表单实例：`form`

```
= ContactForm() 。
```

6. 渲染模板:

- 最后, 视图通过 `render()` 函数将表单渲染到 HTML 页面中, 并将表单对象作为上下文传递:

```
return render(request, 'contact_form.html', {'form': form})
```

。

图片中的附加说明

- 提到可以通过设置表单初始值来定制表单, 比如:

```
form = ContactForm(initial={'subject': 'I like your site!'})
```

。
- 表单清理的数据被描述为字典, 方便开发者处理用户输入。

`render()`

What is the result of render()?

`render(request, template_name, context=None)`

- Combines a given template with a given context dictionary and returns an [HttpResponse](#) object with that rendered text.
- `request`** and **`template_name`** are required arguments.
- Optional argument:
 - `context`**: A dictionary of values to add to the template context. By default, this is an empty dictionary.

`render()` 方法的功能

1. 作用:

它将给定的模板文件 (`template_name`) 和上下文数据 (`context`) 组合, 并生成一个 `HttpResponse` 对象。

这个对象包含渲染后的 HTML 页面, 可以直接作为服务器的响应返回给用户。

2. 方法参数

- `request` (必需参数):
 - 当前 HTTP 请求对象, 用于标识此次请求来源。
- `template_name` (必需参数):
 - 指定渲染的模板文件, 比如 `index.html`。
- `context` (可选参数):

- 传递给模板的变量字典，供模板使用。
- 如果未指定，则默认为空字典。

3. 返回结果

- `render()` 返回的是一个 `HttpResponse` 对象，其中包含渲染后的 HTML 文本。这些内容会显示在用户浏览器上，形成网页。

```
Python |  
1 from django.shortcuts import render  
2 def home_view(request):  
3     context = {'message': 'Welcome to my site!'}  
4     return render(request, 'home.html', context)
```

在这个示例中：

- 模板文件 `home.html` 将被渲染。
- 上下文数据 `{'message': 'Welcome to my site!'}` 将传递给模板进行动态生成。

如何创建 表单页面 & 运行结果



1. HTML模板文件:

通过 Django 的模板系统定义了表单页面 `contact_form.html`。

模板中包含 Django 的模板标签，例如 `{% if form.errors %}`，用于检查表单是否有错误并显示适当的提示。

2. 表单内容:

使用 `{{ form.as_table }}` 渲染表单字段，并将每个字段显示为表格样式。

添加了 `csrf_token`，以确保表单提交的安全性。

使用 `<input type="submit">` 按钮提交表单数据。

3. 表单错误提示:

如果用户输入数据有误，模板会显示一条红色提示信息：“Please correct the error(s) below.” 并指出具体字段的错误。

4. URL配置:

修改 `urls.py` 文件，将 `/contact/` 映射到视图函数，用于显示该联系表单。

Result of running /contact/

Load the form,

- submit it with none of the fields filled out,
- submit it with an invalid e-mail address, then finally
- submit it with valid data.
 - Of course, unless you have configured a mail-server, you will get a `ConnectionRefusedError` when `send_mail()` is called.

The image shows two screenshots of a web browser displaying a 'Contact us' form. The top screenshot shows the form with empty fields for Subject, Your e-mail address, and Message. The bottom screenshot shows the form after submission with error messages. The Subject field has a message 'This field is required.' and the Your e-mail address field has a message 'This 請在電子郵件地址中包含「@」。' and '「abc」未包含「@」'.

31

1. 加载表单页面:

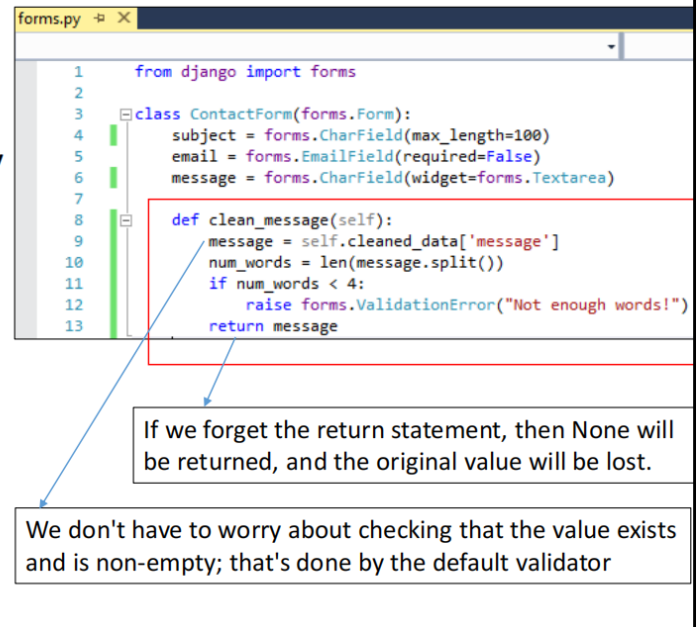
- 用户可以看到一个标题为 "Contact us" 的联系表单页面，包含主题（Subject）、电子邮件地址（Your e-mail address）以及消息内容（Message）的输入字段。

2. 测试提交数据:

- 提交以下三种类型的数据时的结果：
 - 未填写任何字段: 显示错误提示，比如主题字段为必填（“This field is required”）。
 - 输入无效的电子邮件地址: 提示需要有效的电子邮件地址（“Enter a valid e-mail address”）。
 - 填写有效数据并提交: 如果未配置邮件服务器，则会触发 `ConnectionRefusedError`，因为 `send_mail()` 无法正常发送邮件。

Custom validation

- Django's form system automatically looks for any method whose name **starts with clean_ and ends with the name of a field**.
- If any such method exists, it's called during validation.
- Specifically, the `clean_message()` method will be called after the default validation logic for a given field (in this case, the validation logic for a required `CharField`).



1. 自动调用 `clean_` 方法:

- Django 的表单系统会自动寻找名称以 `clean_` 开头, 并以字段名称结尾的方法, 例如 `clean_message()`。
- 如果这种方法存在, 它会在默认验证逻辑之后被调用, 允许开发者为字段添加额外的验证规则。

应用示例: 图片中以 `ContactForm` 为例, 展示了如何为 `message` 字段定义额外验证:

2. python

```
1 def clean_message(self):
2     message = self.cleaned_data['message']
3     num_words = len(message.split())
4     if num_words < 4:
5         raise forms.ValidationError("Not enough words!")
6     return message
```

• 逻辑说明:

方法从 `self.cleaned_data` 中提取经过默认验证的 `message` 数据。

使用 `message.split()` 计算消息中的单词数量。

如果单词数量小于 4, 则抛出 `ValidationError` 提示用户错误。

方法结束时必须返回经过验证的 `message` 值，否则原值会丢失。

3. 默认验证与自定义验证结合:

- 自定义验证逻辑与 Django 的默认验证逻辑结合使用。例如，对于一个必填字段（`CharField`），系统会自动验证字段是否为空，开发者无需重复处理。

注意事项:

1. 返回值的重要性:

如果 `clean_` 方法忘记返回字段的值，默认返回 `None`，导致原数据丢失。

2. `ValidationError`:

用于向用户传递错误信息，通常会显示在表单页面中，帮助用户纠正输入。

实际应用场景:

这种自定义验证非常适合以下场景:

- **内容限制:** 比如要求用户输入一定长度的消息或满足某种格式。
- **复杂逻辑检查:** 验证字段之间的关系或特定业务规则。

Django Forms vs Plain HTML Forms

Advantages of Django forms:

- Django's form class provides validation on many forms. For example, if you create a form and there are many blank fields (by default all fields are required meaning a user must enter in a value), the form will be submitted; the user will be told to fill in the blank fields before the form is submitted.
- Django's form class also provides validation with many different types of data entry. For example, it has an `EmailField` that validates email address. So, if a user types in, 'Peter' as his email, Django's form states that this is not a valid email address.
- Django forms gives ease of use when taking data from a form and inserting the data into a database.

1. 默认字段验证:

Django 的表单类默认会要求所有字段必须填写（除非明确设置为非必填）。

如果用户提交表单时有字段为空，表单验证会失败，并提示用户需要填写这些字段。例如，如果某些必填字段是空白的，系统会通知用户补全。

2. 针对特定数据类型的验证：

- Django 表单类可以验证输入数据是否符合特定类型。比如：
 - **EmailField**：用于验证电子邮件地址的格式。如果用户输入类似 "Peter"，而不是有效的邮箱（如 "Peter@example.com"），Django 表单会提示这是无效的电子邮件地址。
 - 类似的，还有其他字段类型（如 IntegerField、URLField）也具备内置的验证功能。

Advantages of Plain HTML Forms

- Using plain HTML forms, you are not restricted to Django's form class rules, so you have more flexibility to make the form however you want.

Disadvantages of Plain HTML Forms

- So the disadvantages of plain HTML forms is that you have to write all of the validation tools yourself. This includes email validation, integer validation (if working with numbers), etc.